PYBAQ · MEETUP TOOLING

How `raffle-cli` was built

A beginner's guide to the code, the diagrams and the technical decisions behind it

> **Version covered:** 0.2.0 · **Python:** 3.10+ · **Code:** 1109 lines in 9 modules, 43 tests
>
> **Honest note:** the first prototype was written by Andoresu; this version (restructure, fixes, persistence, fireworks, Textual slot machine, tests, docs) was written together with Claude, an AI assistant. Reading this guide is how the learning gets back to you: every important line is explained, every decision has a reason, and the exercises at the end ask you to change the program yourself.

How to use this guide. Read a chapter, then open the file it talks about and find the code. Run the program. Change a number, break something on purpose, read the error. Typing code yourself teaches far more than reading it.

Contents

1 What we built

2 The big picture

3 Where we started: the review

4 Python foundations you meet in this code

5 The raffle engine

6 Files, validation and resuming

7 Randomness, fairness and --seed

8 The classic terminal interface

9 The fireworks

10 The Textual interface

11 The slot machine

12 Testing

13 Technical decisions, in one table

14 Things that went wrong (and what they taught)

15 Exercises

16 Glossary and how to run everything

1 What we built

`raffle-cli` runs a raffle in the terminal for meetup giveaways: it is meant to be *watchable*, so everybody can see the draw is fair. You give it three files (people, prizes, and a config) and it draws the winners one prize at a time.

You can…	With
Create the starter files	`raffle-cli setup`
Run the classic raffle (press Enter to reveal each winner)	`raffle-cli`
Run it as a full-screen slot machine, with mouse support	`raffle-cli --tui`
Watch the fireworks finale on its own	`raffle-cli fireworks`
Make a draw reproducible and auditable	`--seed 7` (a seed is always shown in the results)
Pick the prize order, skip delays, run unattended	`--order`, `--fast`, `--yes`
Recover after a crash	winners are saved the moment they are revealed; run again and it resumes

The three files

config.yaml

```yaml
participants_file: participants.csv
prizes_file: prizes.csv
winners_file: winners.csv
recreate_from_winners: true
```

participants.csv

```
name,lastname
Ada,Lovelace
Grace,Hopper
```

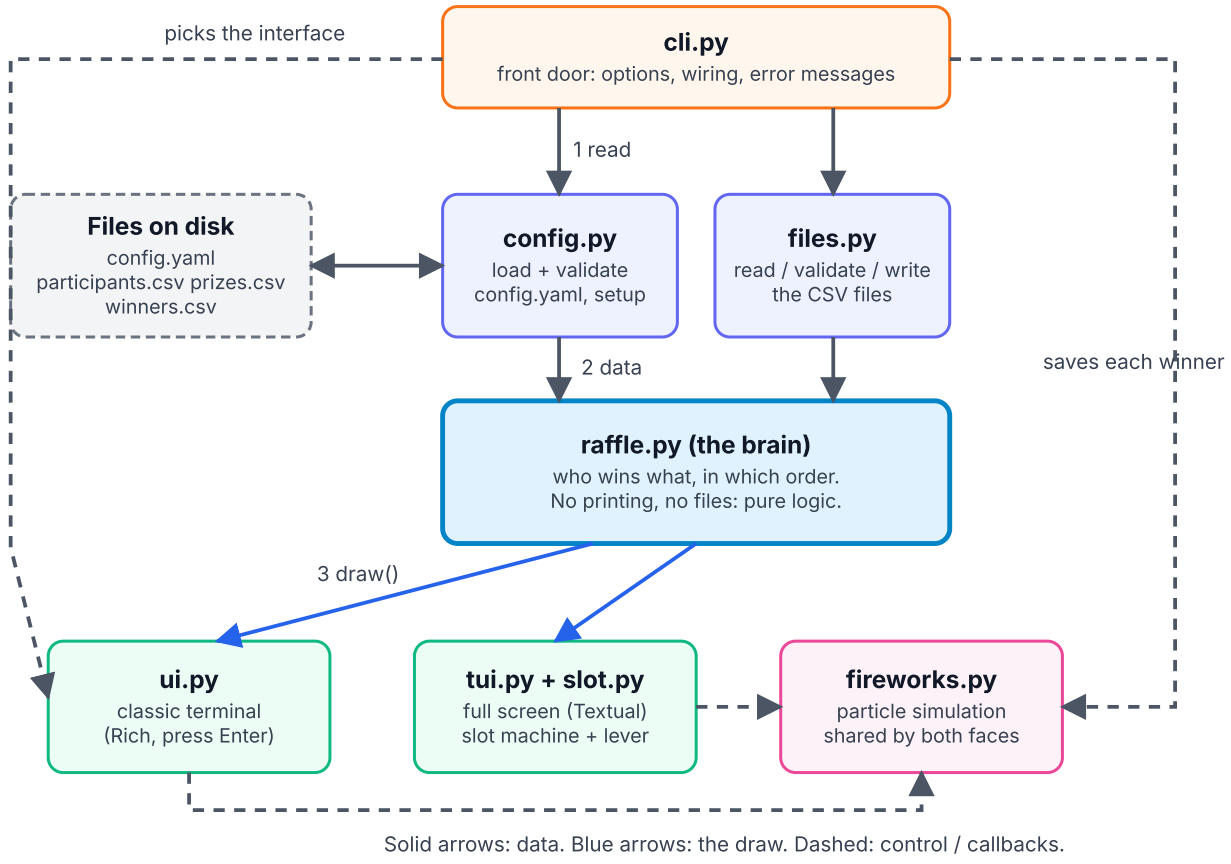
prizes.csv

```
item,qty
Mechanical keyboard,1
Python mug,2
```

The program writes a fourth file, `winners.csv`, with columns `name,lastname,item`.

2 The big picture

The original project was one file with everything mixed together: reading files, drawing winners, and printing. The most important change was to **pull those jobs apart**. Programmers call this *separation of concerns*: each piece does one thing, and you can change or test one piece without touching the others.

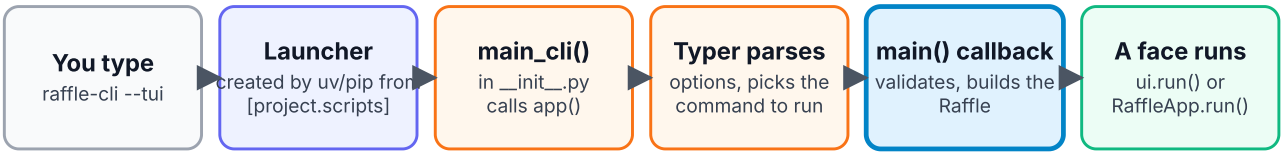


File	Job	Lines	Python ideas you will meet
`raffle.py`	The brain: who wins what. No printing, no files.	131	dataclasses, type hints, tuples as sort keys, `random.Random`
`files.py`	Read, validate and write the CSV files	145	`csv`, `pathlib`, custom exceptions, encodings
`config.py`	Load `config.yaml`, create starter files	77	Pydantic models, YAML, closures
`cli.py`	Front door: options, wiring, friendly errors	143	Typer, `Annotated`, callbacks, exit codes
`ui.py`	The classic interface	99	Rich panels and tables, callbacks
`fireworks.py`	Particle simulation + renderer	197	classes, simple physics, `rich.live.Live`
`tui.py`	Full-screen Textual app	152	widgets, CSS, key bindings, timers, messages

`slot.py`	The slot machine widget	153	custom widget, animation by timers, mouse events

What happens when you type `raffle-cli`

A command like `raffle-cli` is not magic: the packaging tool created a tiny launcher script that imports your function and calls it.



pyproject.toml: [project.scripts] raffle-cli = "raffle_cli:main_cli"

pyproject.toml (excerpt)

```
[project.scripts]
raffle-cli = "raffle_cli:main_cli"

[project.optional-dependencies]
tui = ["textual>=0.86"]
```

3 Where we started: the review

The first step was *not* writing code: it was reading the prototype and actually running it to confirm each suspicion. Bugs you only suspect are opinions; bugs you reproduce are facts. These are the real problems found, and where each is solved.

Problem found (reproduced by running it)	Fix	Chapter
`raffle-cli setup` ran a whole raffle first (the main callback runs before subcommands)	`ctx.invoked_subcommand` check	4
Winners were never saved; `recreate_from_winners` did nothing	append each winner; real resume logic	6
Infinite loop when prizes outnumber participants	draw in rounds: everyone wins once before anyone wins twice	5
`UnboundLocalError` on a bad config	validation raises clear errors, exits with code 1	6
Empty or header-only CSVs were accepted, then crashed	validation reports every problem with line numbers	6
`eval()` used to pick constants; names with `[` broke the output	no `eval`; `rich.markup.escape`	4, 8
`keyboard` dependency never used (needs root on Linux); `rich` used but not declared	fixed `pyproject.toml`	4
One 485-line module, no tests, no CI, no LICENSE, README out of date	modules, tests, CI, docs	2, 12

Habit worth stealing
Before changing code, write down what is wrong and prove it. Half of the “bugs” you suspect turn out to be fine, and the other half are usually worse than you thought.

4 Python foundations you meet in this code

4.1 Packaging: from a folder to a command

The code lives in `src/raffle_cli/`. This is the "src layout": because the package is not at the top level, your tests can only import it after it is installed. That catches a classic mistake (code that works on your laptop only because you happen to be standing in the right folder).

uv is the tool that creates a virtual environment (a private set of installed packages for one project), installs what `pyproject.toml` asks for, and records the exact versions in `uv.lock` so everybody gets the same ones. `uv run pytest` runs a command inside that environment.

> **Technical decision: the Textual interface is an optional extra**
>
> **Alternatives:** make Textual a normal dependency. **Why not:** most people only want the classic interface, and Textual is a big dependency. With an extra (`raffle-cli[tui]`) it is installed only on request. The code imports it lazily, inside the `if tui:` branch, and prints a helpful message if it is missing. **Trade-off:** one more thing for users to remember (`uvx --with textual`).

4.2 Type hints and dataclasses

A **dataclass** is a class whose main job is to hold data. Python writes `__init__`, `__repr__` and `__eq__` for you. `frozen=True` makes instances read-only, which also makes them comparable and usable as dictionary keys. The text after the colon (`str`, `int`) is a *type hint*: Python does not enforce it, but your editor and readers do use it.

src/raffle_cli/raffle.py · Participant

```python
@dataclass(frozen=True)
class Participant:
    name: str
    lastname: str

    @property
    def full_name(self) -> str:
        return f"{self.name} {self.lastname}"
```

src/raffle_cli/raffle.py · Prize

```python
@dataclass(frozen=True)
class Prize:
    item: str
    qty: int
```

src/raffle_cli/raffle.py · Winner

```python
@dataclass(frozen=True)
class Winner:
    name: str
    lastname: str
    item: str

    @property
    def full_name(self) -> str:
        return f"{self.name} {self.lastname}"
```

Technical decision: small frozen dataclasses instead of dictionaries

The prototype passed `{"name": ..., "lastname": ...}` dictionaries around. A typo like `row["lasname"]` only fails when that line runs. With a dataclass, `p.lasname` is flagged by your editor immediately, tests can compare whole objects with `==`, and `frozen` guarantees nobody modifies a prize by accident. **Alternative:** Pydantic models (more features, more weight; we use Pydantic only where it earns its keep: validating `config.yaml`).

The line `from __future__ import annotations` at the top of each module makes type hints lazy strings, so newer syntax like `list[str] | None` is safe on Python 3.10.

4.3 pathlib, csv and encodings

`pathlib.Path` treats file paths as objects: `base / "winners.csv"` joins paths correctly on Windows and Linux. The `csv` module handles quoting for you (a prize called `Mug, large` stays one cell). One real-world detail: Excel saves CSVs with an invisible *byte order mark* at the start. We open files with `encoding="utf-8-sig"`, which silently removes it, otherwise the first column would be called `\ufeffname` and validation would fail for a mysterious reason.

4.4 Exceptions that carry information

src/raffle_cli/files.py · DataFileError

```python
class DataFileError(Exception):
    """A work file is missing or invalid; `problems` lists everything wrong with it."""

    def __init__(self, path: Path, problems: list[str]) -> None:
        self.path = path
        self.problems = problems
        super().__init__(f"{path.name}: " + "; ".join(problems))
```

An exception is just a class. This one stores *all* the problems found in a file, not only the first. A person fixing a spreadsheet wants to see every wrong row at once. The command line catches it and prints a bullet list, so users never see a traceback for their own typos.

4.5 Pydantic for the config file

src/raffle_cli/config.py · RaffleConfig

```python
class RaffleConfig(BaseModel):
    participants_file: Path = Path("participants.csv")
    prizes_file: Path = Path("prizes.csv")
    winners_file: Path = Path("winners.csv")
    # True: resume from whatever is already in winners_file. False: start fresh each run.
    recreate_from_winners: bool = True

    def resolved(self, base: Path) -> "RaffleConfig":
        """Make relative file paths relative to the config file's folder."""
        return RaffleConfig(
            participants_file=base / self.participants_file,
            prizes_file=base / self.prizes_file,
            winners_file=base / self.winners_file,
            recreate_from_winners=self.recreate_from_winners,
        )
```

Pydantic checks the shape of data using the type hints. If someone writes `recreate_from_winners: maybe`, Pydantic raises `ValidationError`, which `load_config` turns into one readable message. The paths are resolved relative to the config file's folder (`resolved()`), so `raffle-cli -c event/config.yaml` works from anywhere.

4.6 Typer: turning functions into commands

Typer reads a function's parameters and builds the command line from them. `@app.command()` adds a subcommand (`setup`, `fireworks`); `@app.callback(invoke_without_command=True)` defines code that runs for `raffle-cli` itself and holds the global options.

`src/raffle_cli/cli.py · the main callback (start)`

```
@app.callback(invoke_without_command=True)
def main(
    ctx: typer.Context,
    config: ConfigOption = None,
    seed: Annotated[Optional[int], typer.Option(help="Seed the draw so it can be reproduced. Shown in
the results.")] = None,
    order: Annotated[str, typer.Option(help=f"Order prizes are raffled: {', '.join(ORDERS)}.")] = "high-
low",
    fast: Annotated[bool, typer.Option("--fast", help="Skip the suspense delays.")] = False,
    yes: Annotated[bool, typer.Option("--yes", "-y", help="Never wait for Enter (non-interactive).")] =
False,
    fireworks_flag: Annotated[bool, typer.Option("--fireworks/--no-fireworks", help="Finish with
fireworks.")] = True,
    tui: Annotated[bool, typer.Option("--tui", help="Use the full-screen Textual interface.")] = False,
    version: Annotated[bool, typer.Option("--version", callback=version_callback, is_eager=True,
help="Show the version.")] = False,
) -> None:
    """Run the raffle (or a subcommand)."""
    ctx.obj = {"config": config}
```

`… and what it does next`

```
if ctx.invoked_subcommand is not None:
    return
if order not in ORDERS:
    raise fail(f"--order must be one of: {', '.join(ORDERS)}.")
seed = secrets.randbits(32) if seed is None else seed
raffle, cfg = build_raffle(config or DEFAULT_CONFIG, seed, order, interactive=not yes)
```

> **Technical decision: a guard in the callback instead of a separate "run" command**
>
> The prototype's callback ran the raffle *always*, even for `raffle-cli setup`. The fix is two lines: if Typer says a subcommand was requested (`ctx.invoked_subcommand`), return immediately. **Alternative:** a dedicated `raffle-cli run`; cleaner in theory but it would break the simple, memorable `raffle-cli` from the original README. **Trade-off:** options like `--seed` live on the callback, so they go *before* a subcommand name.

`Annotated[bool, typer.Option("--fast", help="…")] = False` reads as: "a bool, shown as the option `--fast` with this help text, default False". Errors go through one helper, `fail()`, which prints in red and returns `typer.Exit(1)`; the non-zero exit code lets scripts detect failure.

4.7 Rich: colour without the pain

Rich prints styled text with a tiny markup language: `[bold green]hi[/]`. The catch: user data can contain square brackets. A participant called `[bold]Eve` would be interpreted as markup. Every

place that prints names uses `rich.markup.escape()`.

5 The raffle engine

`raffle.py` is the heart. It imports no UI library, prints nothing and touches no files. That is why it is easy to test and why *two* different interfaces (classic and Textual) can share it without duplicating a line of draw logic.

5.1 The data model: a pile of prize units

src/raffle_cli/raffle.py · Raffle.__init__

```
def __init__(
    self,
    participants: Sequence[Participant],
    prizes: Sequence[Prize],
    *,
    previous_winners: Iterable[Winner] = (),
    rng: random.Random | None = None,
    order: str = "high-low",
) -> None:
    if not participants:
        raise ValueError("A raffle needs at least one participant.")
    self.rng = rng or random.Random()
    self.participants = list(participants)
    self.winners: list[Winner] = []
    self._wins = [0] * len(self.participants)
    self._held: list[set[str]] = [set() for _ in self.participants]
    self._units: list[str] = [
        p.item for p in order_prizes(prizes, order, self.rng) for _ in range(p.qty)
    ]
    for winner in previous_winners:
        self._restore(winner)
```

Instead of tracking “Mug: 2 left”, the constructor expands every prize into *one entry per physical item*: `["Mug", "Mug", "Pen", ...]`. Giving a prize away is just `pop(0)`. Counting remaining prizes is counting the list. Fewer special cases means fewer bugs.

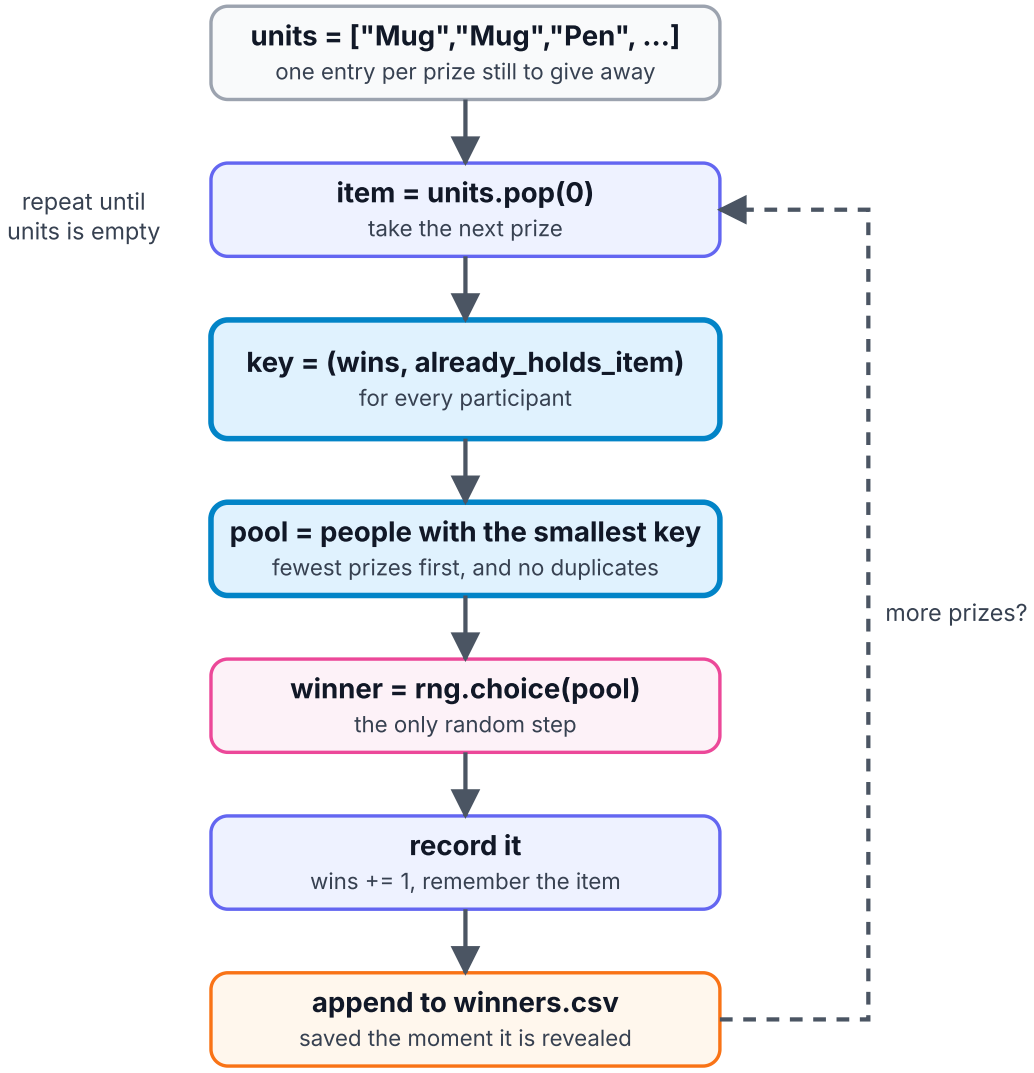
src/raffle_cli/raffle.py · order_prizes

```
def order_prizes(prizes: Sequence[Prize], order: str, rng: random.Random) -> list[Prize]:
    """Return a new list of prizes sorted by `order` ('high-low', 'low-high', 'shuffle' or 'file')."""
    ordered = list(prizes)
    if order == "shuffle":
        rng.shuffle(ordered)
    elif order == "low-high":
        ordered.sort(key=lambda p: p.qty)
    elif order == "high-low":
        ordered.sort(key=lambda p: p.qty, reverse=True)
    elif order != "file":
        raise ValueError(f"Invalid order {order!r}. Choose from {', '.join(ORDERS)}.")
    return ordered
```

Technical decision: never modify the caller's data

The prototype decremented `prize["qty"]` in place, so the same list could not be reused (for a second run, a test, or a re-render). `order_prizes` returns a *new* list; `Prize` is frozen; the engine keeps its own `_units`. A test (`test_prizes_are_not_mutated`) pins this down.

5.2 How a winner is chosen



src/raffle_cli/raffle.py · Raffle.draw

```
def draw(self) -> Winner:
    if self.done:
        raise RuntimeError("All prizes have been awarded.")
    item = self._units.pop(0)
    keys = [(self._wins[i], item in self._held[i]) for i in range(len(self.participants))]
    best = min(keys)
    pool = [i for i, key in enumerate(keys) if key == best]
    return self._record(self.rng.choice(pool), item)
```

The core idea is one line: `keys = [(wins, holds_item) ...]`. Python compares tuples element by element, so `(0, False)` sorts before `(1, False)`, which sorts before `(1, True)`. `min(keys)` gives the “most deserving” key; `pool` is everyone sharing it; `rng.choice(pool)` is the single random step. The rules fall out of the tuple:

- **Nobody wins twice before everybody has won once** (first element: fewest wins).
- **In later rounds, avoid giving someone an item they already hold** (second element), unless there is no alternative.

A worked example

Three people (Ana, Ben, Cy), prizes Mug×2, Pen×2, Sticker×1. `high-low` ordering (stable) gives `units = [Mug, Mug, Pen, Pen, Sticker]`. One possible run:

#	Prize	Pool (smallest key)	Winner	Wins after (Ana, Ben, Cy)
1	Mug	Ana, Ben, Cy: all (0, no)	Ben	0, 1, 0
2	Mug	Ana, Cy: still at 0 wins	Cy	0, 1, 1
3	Pen	Ana: the only one with 0 wins	Ana	1, 1, 1
4	Pen	Ben, Cy: Ana is (1, **holds Pen**) so she is behind them	Cy	1, 1, 2
5	Sticker	Ana, Ben: fewest wins (1)	Ben	1, 2, 2

Five prizes, three people: nobody got a second prize until draw 4, because only then had everyone won once. That is exactly the rule "if there are more prizes than participants, people get more than one".

> **Technical decision: rounds instead of "pick anyone, then swap"**
>
> The prototype picked a random person with replacement; if they already had a prize it asked "swap?". That made the raffle interactive in a confusing way, and it looped forever when prizes outnumbered people, because swapping never reduced the pile. **Our rule:** draw from the people with the fewest wins. It is simple to explain to an audience, can never loop forever, and is provably fair (the tests assert that with 10 prizes and 5 people everybody gets exactly two). **Trade-off:** we dropped the swap prompt. It could come back as an option.

> **Technical decision: participants are identified by position, not by name**
>
> Two people can share a name. If winners were looked up by `(name, lastname)`, they would be merged into one. The engine keeps `_wins` and `_held` as lists indexed by participant position, so duplicates are two separate chances. The CSV reader only prints a *warning* for duplicates.

5.3 Rebuilding state: resuming

src/raffle_cli/raffle.py · Raffle._restore

```python
def _restore(self, winner: Winner) -> None:
    try:
        self._units.remove(winner.item)
    except ValueError:
        raise ValueError(
            f"Winners file lists '{winner.item}' for {winner.full_name}, "
            "but there are no such prizes left in the prizes file."
        ) from None
    candidates = [
        i for i, p in enumerate(self.participants)
        if p.name == winner.name and p.lastname == winner.lastname
    ]
    if not candidates:
        raise ValueError(f"Winner {winner.full_name} is not in the participants file.")
    i = min(candidates, key=lambda c: self._wins[c])
    self._record(i, winner.item)
```

When resuming, each saved winner is replayed: remove that prize from the pile and count the win. If the saved file contradicts the prizes file (an item that no longer exists, a person who is gone) we raise a clear `ValueError` instead of guessing.

6 Files, validation and resuming

6.1 Reading a CSV defensively

The rule for anything a human types: **assume it is wrong and explain exactly how**. All three readers share one helper that checks the header and returns the rows with their line numbers.

src/raffle_cli/files.py · _read_rows

```
def _read_rows(path: Path, columns: tuple[str, ...]) -> list[tuple[int, dict[str, str]]]:
    """Read `path`, check its header and return (line_number, stripped_row) pairs."""
    if not path.exists():
        raise DataFileError(path, [f"file not found ({path}). Run `raffle-cli setup` to create it."])
    with open(path, newline="", encoding="utf-8-sig") as fh:
        reader = csv.DictReader(fh)
        header = [h.strip().lower() for h in (reader.fieldnames or [])]
        if not header:
            raise DataFileError(path, [f"missing header row, expected: {','.join(columns)}"])
        if set(header) != set(columns) or len(header) != len(columns):
            raise DataFileError(
                path, [f"expected columns {','.join(columns)} but found {','.join(header)}"]
            )
        reader.fieldnames = header
        rows = []
        for row in reader:
            clean = {k: (v or "").strip() for k, v in row.items() if k is not None}
            if any(clean.values()):
                rows.append((reader.line_num, clean))
    return rows
```

src/raffle_cli/files.py · read_prizes

```
def read_prizes(path: Path) -> list[Prize]:
    problems: list[str] = []
    prizes: list[Prize] = []
    for line, row in _read_rows(path, PRIZE_COLUMNS):
        if not row["item"]:
            problems.append(f"line {line}: item is required")
            continue
        try:
            qty = int(row["qty"])
        except ValueError:
            problems.append(f"line {line}: qty must be a whole number, got '{row['qty']}'")
            continue
        if qty < 1:
            problems.append(f"line {line}: qty must be at least 1, got {qty}")
            continue
        prizes.append(Prize(row["item"], qty))
    if not problems and not prizes:
        problems.append("no prizes found")
    if problems:
        raise DataFileError(path, problems)
    return prizes
```

Technical decision: collect every problem, and never “repair” the user's files

Two choices here.

1. Report all problems at once. Failing on the first bad row means ten edit-run cycles for ten typos. The reader gathers a list and raises once.

2. No automatic fixing. The prototype offered “want to set the default content?”, which would overwrite a real participant list if you pressed `y` by reflex. Now a broken file produces an error message and is left untouched; `setup` only writes files that do not exist (or when you pass `--force`).

Smaller details: headers are lower-cased and stripped, cells are stripped, blank lines are ignored, and `qty` must be a whole number ≥ 1.

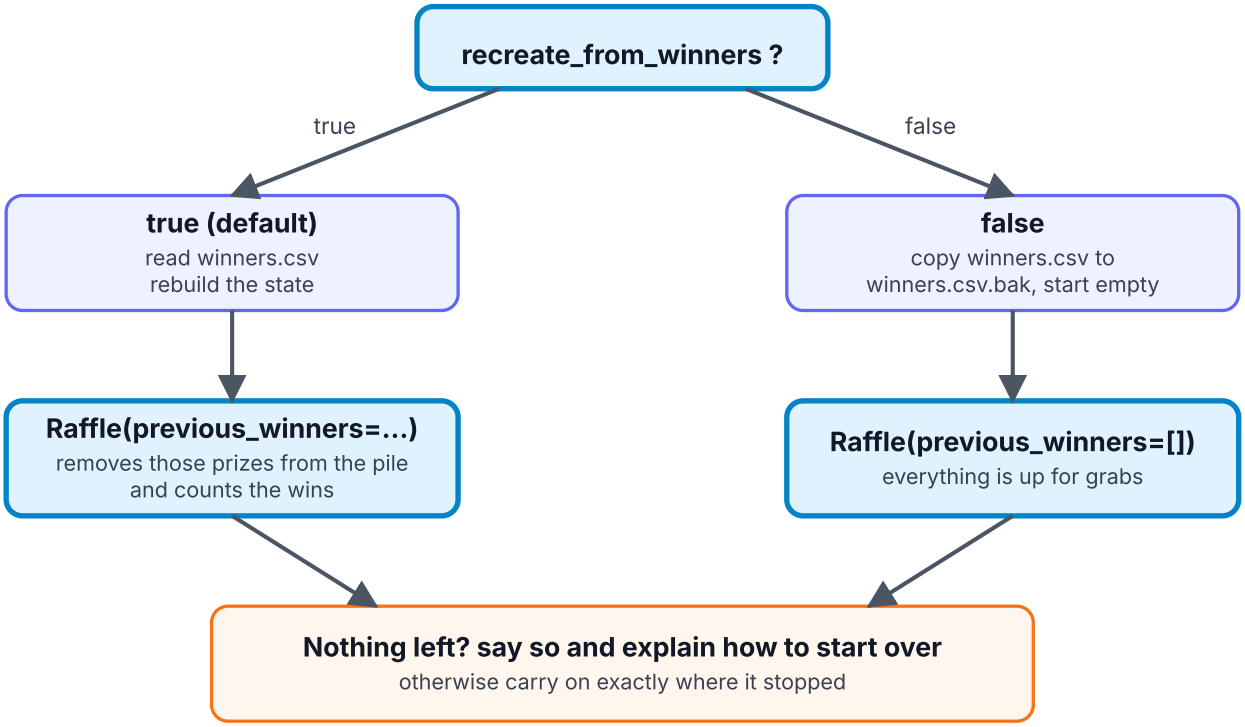
6.2 Saving every winner immediately

src/raffle_cli/files.py · append_winner

```python
def append_winner(path: Path, winner: Winner) -> None:
    """Persist one winner immediately, so a crash mid-event loses nothing."""
    if not path.exists():
        reset_winners(path)
    with open(path, "a", newline="", encoding="utf-8") as fh:
        csv.writer(fh).writerow((winner.name, winner.lastname, winner.item))
```

If the laptop dies in the middle of a meetup, you should not lose the winners already announced. So a winner is appended to `winners.csv` at the moment they are revealed, not at the end. Appending one row is also cheap and safe.

6.3 Starting fresh or resuming



Technical decision: a draw is decided first, saved when revealed

`raffle.draw()` picks the winner *before* the suspense, then the interface plays the animation, and only when the name is revealed do we call the `on_winner` callback that writes the CSV. **Why:** the UI can show anything it likes while the result is already fixed (the slot machine reels simply land on it), and the file never contains a winner the audience has not seen yet. **Trade-off:** if you quit during a spin, that prize is not recorded and will be drawn again on resume (possibly someone else): this is the right behaviour for a live event.

Technical decision: old results are backed up, not erased

With `recreate_from_winners: false` the program starts empty, but first copies a non-empty `winners.csv` to `winners.csv.bak` (see `reset_winners`). Deleting results silently is the kind of mistake you only make once, in front of an audience.

7 Randomness, fairness and --seed

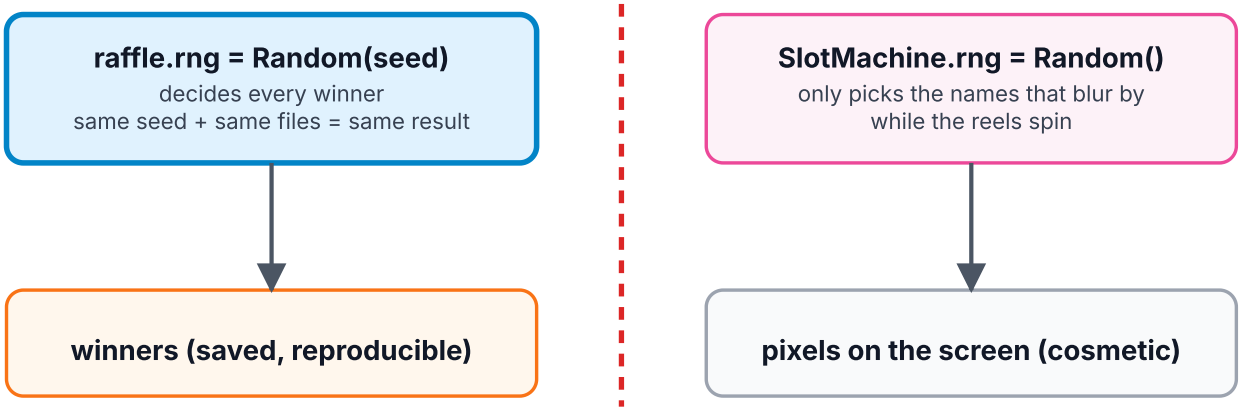
Computers do not produce truly random numbers; `random.Random(seed)` is a *pseudo-random generator*: give it the same seed and it produces the same sequence forever. That sounds like a flaw, but for a transparent raffle it is a feature.

- If no seed is given, one is generated with `secrets.randbits(32)` and **printed in the results**.
- Anyone with the same files and the same `--seed` can reproduce the exact same winners.

> **Technical decision: show the seed instead of hiding randomness**
>
> **Alternative:** `random.SystemRandom()`, which draws from the operating system's entropy: unpredictable, but nobody can ever verify a result. For giving away stickers, *auditability* beats secrecy. A 32-bit seed is not cryptographically strong and that is fine: it is a receipt, not a secret. (If prizes had real money value you would want to commit to the seed publicly before the draw.)

7.1 A bug we found by thinking about it

The first Textual version made the "blurring names" effect by calling `raffle.rng.choice(...)`. Every call consumes a number from the generator, so the spinning animation changed *who won the next prize*: the same seed would give different winners in the classic interface and in the Textual one, and with `--fast` (fewer animation steps) vs normal speed. The fix: the slot machine owns a separate generator.



never shared: animation can't change who wins

> **Rule of thumb**
>
> Anything that must be reproducible gets its own random generator, and nothing cosmetic is allowed to touch it. A test (`test_tui_animation_leaves_the_seeded_draw_untouched`) now guards this.

8 The classic terminal interface

`ui.py` is the first "face". It receives a ready `Raffle` and a `on_winner` function and does presentation only.

src/raffle_cli/ui.py · run

```python
def run(
    raffle: Raffle,
    console: Console,
    on_winner: Callable[[Winner], None],
    *,
    seed: int | None = None,
    fast: bool = False,
    auto: bool = False,
    show_fireworks: bool = True,
) -> None:
    """Run the raffle until every prize is awarded."""
    word_delay = 0.0 if fast else 0.3
    if not raffle.done:
        delayed(console, "Game starting in...", word_delay)
        delayed(console, "3 2 1 Go", 0 if fast else 1)
        time.sleep(0 if fast else 0.5)
    while not raffle.done:
        item = raffle.next_prize
        title(console, "[purple]ONGOING RAFFLE[/]", "raffle-cli", "Let's Raffle!")
        console.print(Panel(Columns([prizes_table(raffle), winners_table(raffle.winners)])))
        delayed(console, f"Next item to win is... {item}", word_delay)
        winner = raffle.draw()
        delayed(console, f"\tThe winner of {item} is.... ", word_delay)
        pause(console, "\tPress enter when ready.", auto)
        console.print(f"\t\t[bold green]{escape(winner.full_name)}[/]")
        on_winner(winner)
        if not raffle.done:
            pause(console, "Press enter to continue...", auto)

    show_results(console, raffle.winners, seed)
    if show_fireworks:
        pause(console, "\nPress enter for the finale...", auto)
        captions = [f"{w.full_name}  ·  {w.item}" for w in raffle.winners]
        fireworks.play(console, seconds=3 if fast else 10, captions=captions)
        show_results(console, raffle.winners, seed)
```

Notice what it does *not* know: how winners are chosen (`raffle.draw()`) or where they are saved (`on_winner`). Passing a function as an argument like this is called a **callback**; it keeps the interface and the storage independent.

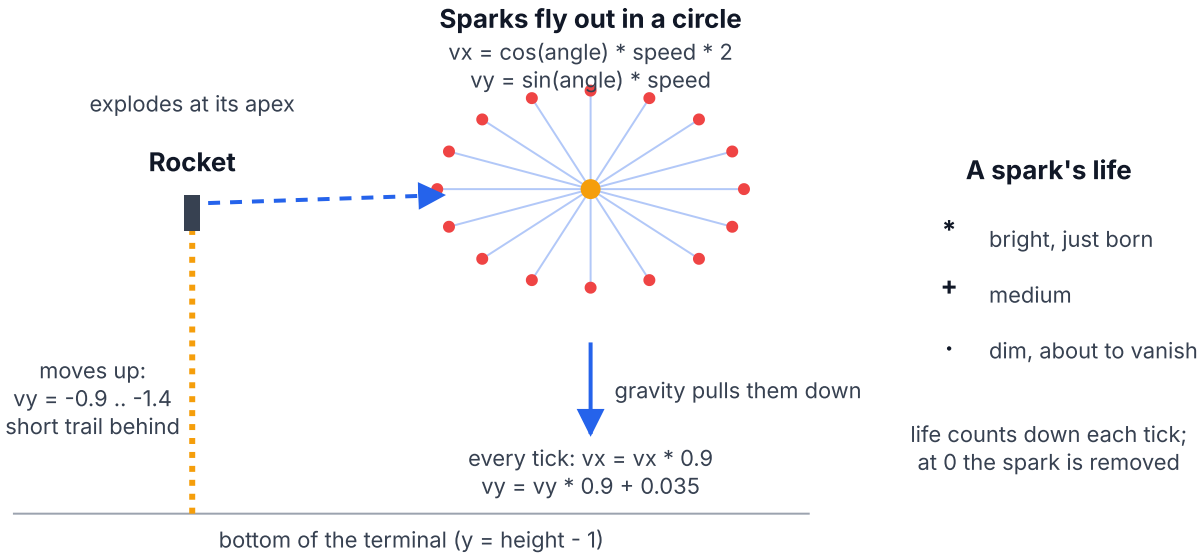
- `delayed()` prints word by word with `time.sleep` for suspense; `--fast` sets the delay to zero.
- `pause()` waits for Enter, but catches `EOFError` so the program does not crash when input is piped (for example `echo | raffle-cli` in a script).
- The same `Raffle` object is shown in tables via `raffle.remaining` and `raffle.winners`.

src/raffle_cli/ui.py · pause

```python
def pause(console: Console, prompt: str, auto: bool) -> None:
    if auto:
        return
    try:
        console.input(f"[dim]{prompt}[/]")
    except EOFError:  # piped stdin: carry on instead of crashing
        pass
```

9 The fireworks

Fireworks are a classic first *simulation*: keep a list of moving things, update each one a little per "tick", draw them all, repeat 20 times a second. There are two kinds of things: **rockets** (go up, then explode) and **sparks** (fly out, slow down, fall, fade).



src/raffle_cli/fireworks.py · Fireworks.step

```python
def step(self) -> None:
    self.tick += 1
    if self.rng.random() < self.launch_rate:
        self._launch()
    for rocket in self.rockets[:]:
        rocket.y += rocket.vy
        self.sparks.append(Spark(rocket.x, rocket.y + 1, 0, 0, 5, "orange3"))
        if rocket.y <= rocket.apex:
            self.rockets.remove(rocket)
            self._explode(rocket)
    for s in self.sparks[:]:
        s.x += s.vx
        s.y += s.vy
        s.vx *= DRAG
        s.vy = s.vy * DRAG + GRAVITY
        s.life -= 1
        if s.life <= 0 or not (-1 <= s.x <= self.width and -1 <= s.y <= self.height):
            self.sparks.remove(s)
```

src/raffle_cli/fireworks.py · Fireworks._explode

```python
def _explode(self, rocket: Rocket) -> None:
    rng = self.rng
    count = rng.randint(30, 55)
    shape = rng.choice(["ring", "burst"])
    colors = [rocket.color] if rng.random() < 0.6 else [rocket.color, rng.choice(PALETTE)]
    ttl = rng.randint(14, 24)
    for i in range(count):
        angle = 2 * math.pi * i / count
        speed = 0.75 if shape == "ring" else rng.uniform(0.25, 0.8)
        self.sparks.append(
            Spark(
                rocket.x,
                rocket.y,
                math.cos(angle) * speed * ASPECT,
                math.sin(angle) * speed,
                ttl + rng.randint(-3, 3),
                rng.choice(colors),
            )
        )
```

Three small bits of maths do the work:

- **Angle:** `2π · i / count` spreads the sparks evenly around a circle. “ring” bursts all use the same speed; “burst” bursts use random speeds.
- **Aspect ratio:** terminal cells are about twice as tall as wide, so horizontal speed is multiplied by `ASPECT = 2`. Without it, circles look like tall ovals.
- **Drag and gravity:** each tick the speed is multiplied by 0.9 (air resistance) and gravity adds 0.035 to the downward speed. A spark with speed 0.75 travels about `0.75 / (1 − 0.9) ≈ 7.5` rows before stopping, which is just right for a terminal.

9.1 Drawing a frame

`render()` makes an empty grid of characters, “plots” every rocket and spark into it (the character depends on how much life is left: `*`, `+`, `.`), overlays the title and the winner caption, and returns one `rich.text.Text`. Note that `Fireworks` never prints anything itself.

src/raffle_cli/fireworks.py · play

```python
def play(
    console: Console,
    *,
    seconds: float = 8.0,
    fps: int = 20,
    title: str = "CONGRATULATIONS",
    captions: list[str] | tuple[str, ...] = (),
    rng: random.Random | None = None,
) -> None:
    """Run the show full-screen. Ctrl+C skips it. Does nothing when not attached to a terminal."""
    if not console.is_terminal:
        return
    width, height = console.size
    show = Fireworks(width, height, rng, title=title, captions=captions)
    for _ in range(8):
        show._launch()
    try:
        with Live(console=console, screen=True, auto_refresh=False) as live:
            for _ in range(int(seconds * fps)):
                show.resize(*console.size)
                show.step()
                live.update(show.render(), refresh=True)
                time.sleep(1 / fps)
```

```
    except KeyboardInterrupt:
        pass
```

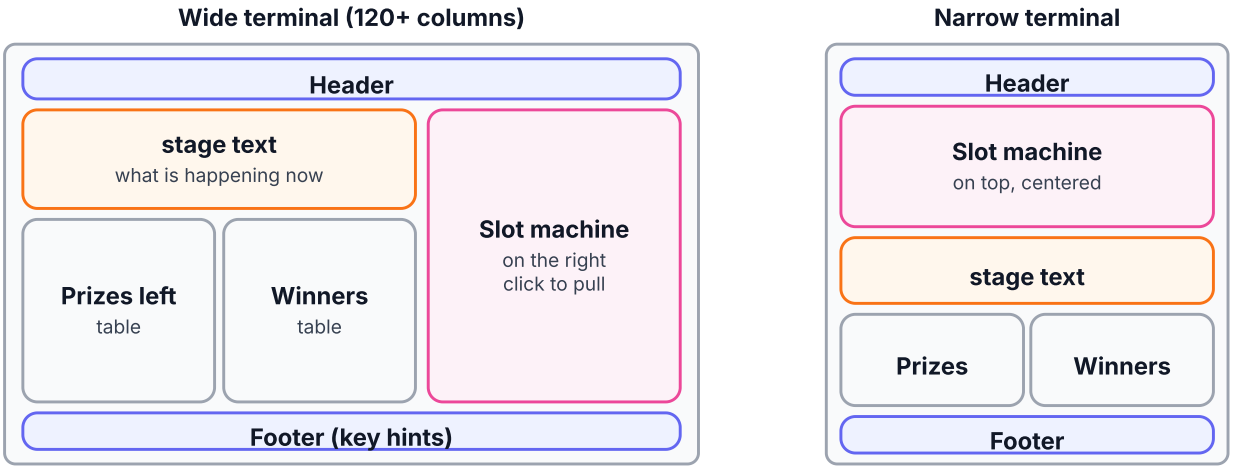
Technical decision: one simulation, two renderers

`Fireworks` only produces a `Text`. `play()` shows it with Rich's `Live(screen=True)` (the terminal's "alternate screen", restored afterwards, like `vim` or `less`), and the Textual interface shows the same object inside a `Static` widget refreshed by a timer. **Alternatives:** a curses program (not portable to Windows) or a fireworks library (a dependency for 200 lines of code). **Trade-off:** we implement the physics ourselves, which is also more fun to learn from. It does nothing when not attached to a terminal (`console.is_terminal`), so scripts and tests do not hang.

10 The Textual interface

Textual is a framework for full-screen terminal apps with ideas borrowed from web pages: *widgets* (tables, labels, your own), layout described with *CSS*, *events* (key, mouse, resize), *timers*, and *messages* that widgets send to each other.

10.1 Layout that adapts to the window



One CSS class on #main ("wide") flips between the two layouts when the window resizes.

src/raffle_cli/tui.py · constants and CSS

```
WIDE_AT = 120  # columns; at least this wide the slot machine moves to the right


class RaffleApp(App):
    TITLE = "raffle-cli"
    CSS = """
    #main { height: 1fr; }
    #machine-box { height: 8; width: 100%; align: center middle; }
    #main.wide #machine-box { dock: right; width: 52; height: 100%; }
    #stage { height: 6; content-align: center middle; text-align: center; border: round $accent; margin:
0 1; }
    #tables { height: 1fr; }
    #tables > Vertical { width: 1fr; padding: 0 1; }
    """
    BINDINGS = [
```

src/raffle_cli/tui.py · RaffleApp._layout

```
def _layout(self, width: int) -> None:
    self.query_one("#main").set_class(width >= WIDE_AT, "wide")
```

The slot machine sits in a container (`#machine-box`) that is, by default, a full-width row at the top. When the window is at least 120 columns wide, Python adds the CSS class `wide` to `#main` , and the rule `#main.wide #machine-box { dock: right; ... }` moves it to the right edge. Python decides *when*; CSS decides *how it looks*. Resizing is handled by `on_resize` calling `_layout` again.

10.2 Key bindings and a surprising bug

src/raffle_cli/tui.py · RaffleApp.action_draw

```
def action_draw(self) -> None:
    if self._busy or self.raffle.done:
        return
    self._busy = True
    item = self.raffle.next_prize
    winner = self.raffle.draw()  # decided now; the reels just land on it
    self._stage(f"Drawing [b yellow]{escape(item)}[/]\n\n[dim]Good luck![/]")

    def reveal() -> None:
        self.on_winner(winner)
        self._refresh()
        hint = "That was the last prize!" if self.raffle.done else "Press Enter or click the machine for
the next prize"
        self._stage(
            f"[b yellow]{escape(item)}[/] goes to\n\n[b green]🎉 {escape(winner.full_name)} 🎉
[/]\n\n[dim]{hint}[/]"
        )
        self._busy = False
        if self.raffle.done and self.show_fireworks:
            self.set_timer(0.5 if self.fast else 3.0, self.action_fireworks)

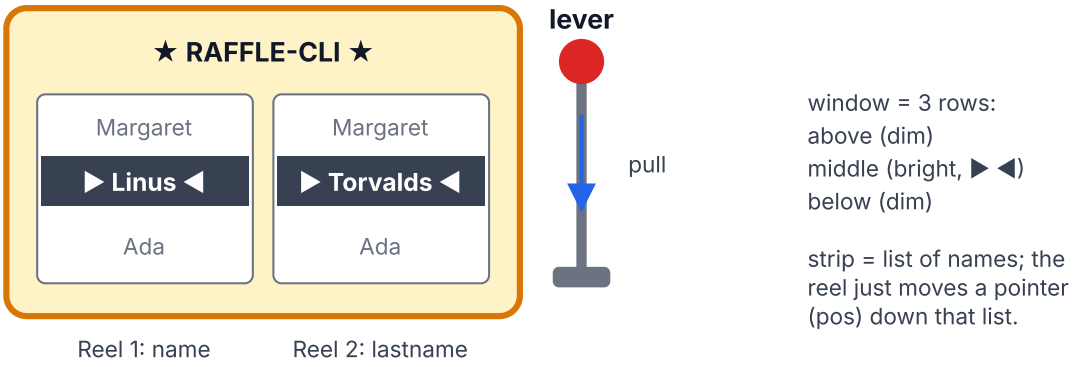
    self.query_one(SlotMachine).pull(winner, reveal)
```

`BINDINGS` maps keys to *actions* (methods named `action_...`). Our first version did nothing when you pressed Enter, because the focused `DataTable` widget handles Enter itself ("select this row") before the app sees it. The fix is `priority=True` on the binding, which lets the app win. The test suite caught this: it presses Enter and checks that a prize was drawn.

> **Technical decision: the app tells widgets what to do; widgets send messages back**
>
> Notice `action_draw` calls `machine.pull(winner, reveal)` (the app commands the widget), while the widget tells the app "I was clicked" by *posting a message* (`Pulled`). The app never reaches into the widget's internals and the widget knows nothing about raffles beyond a winner to land on. This is the same callback idea as in chapter 8.

11 The slot machine

The slot machine is a **custom widget**: a class that inherits from Textual's `Widget` and implements `render()`, returning what to draw. Everything you see is text: box-drawing characters, a few colours, and the lever made of `●`, `┃` and `┻`.



A **reel** is just a list of names (the strip) and a number (`pos`) saying which one is in the middle row. "Spinning" means increasing `pos` and re-drawing. To make the reel land on the winner, the strip is built *backwards*: random names, with the winner placed at index `steps`, and the reel stops when `pos` reaches it. The names directly above and below the winner are re-rolled so you never see the winner's name twice.

src/raffle_cli/slot.py · SlotMachine._start_reels

```python
def _start_reels(self, winner: Winner) -> None:
    self._pending = len(self.reels)
    for i, part in enumerate((winner.name, winner.lastname)):
        steps = (3 if self.fast else 18) + i * (2 if self.fast else 12)  # the right reel spins longer
        strip = self._random_strip(i, steps + 2)
        strip[steps] = part
        for neighbour in (steps - 1, steps + 1):  # never show the winner's own name next to it
            alternatives = [x for x in self.pools[i] if x != part]
            if strip[neighbour] == part and alternatives:
                strip[neighbour] = self.rng.choice(alternatives)
        reel = self.reels[i]
        reel.strip, reel.pos, reel.target, reel.landed = strip, 1, steps, False
        self._advance(i)
```

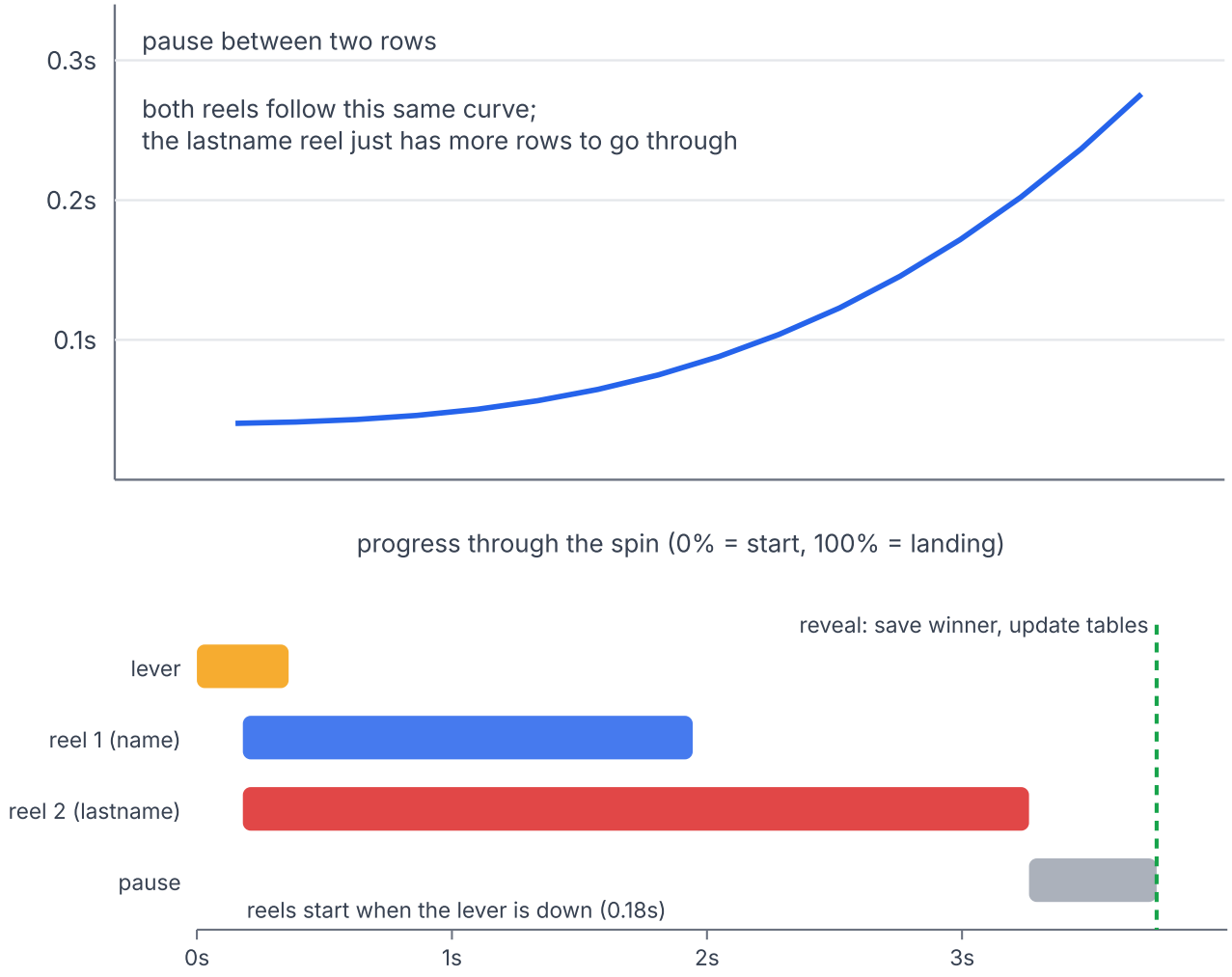
11.1 Animation without a loop: timers

You might expect a `while` loop with `sleep` inside. That would freeze the whole app, because Textual needs to keep handling keys, the mouse and drawing. Instead, each step does a little work and then asks Textual, "call me again in N seconds" with `set_timer`. Between calls, Textual is free to do other things.

src/raffle_cli/slot.py · SlotMachine._advance

```python
def _advance(self, i: int) -> None:
    reel = self.reels[i]
    reel.pos += 1
    if reel.pos >= reel.target:
        reel.landed = True
        self._pending -= 1
        self.refresh()
        if self._pending == 0 and self._on_done:
            self.set_timer(0.05 if self.fast else 0.5, self._on_done)
        return
    self.refresh()
    # slow down like a real machine: 40ms per row at first, ~320ms for the last rows
    delay = 0.02 if self.fast else 0.04 + 0.28 * (reel.pos / reel.target) ** 3
    self.set_timer(delay, lambda: self._advance(i))
```

The delay grows with the cube of the progress: `0.04 + 0.28 · (pos/steps)³`. Early on, rows fly by every 40 ms; near the end the gaps stretch to about 0.3 s, exactly like a real machine slowing down. A cube keeps it fast for a long time and then brakes hard. The right reel has 12 more rows to travel, so it lands later: the classic “left reel stops, then the right one” suspense.



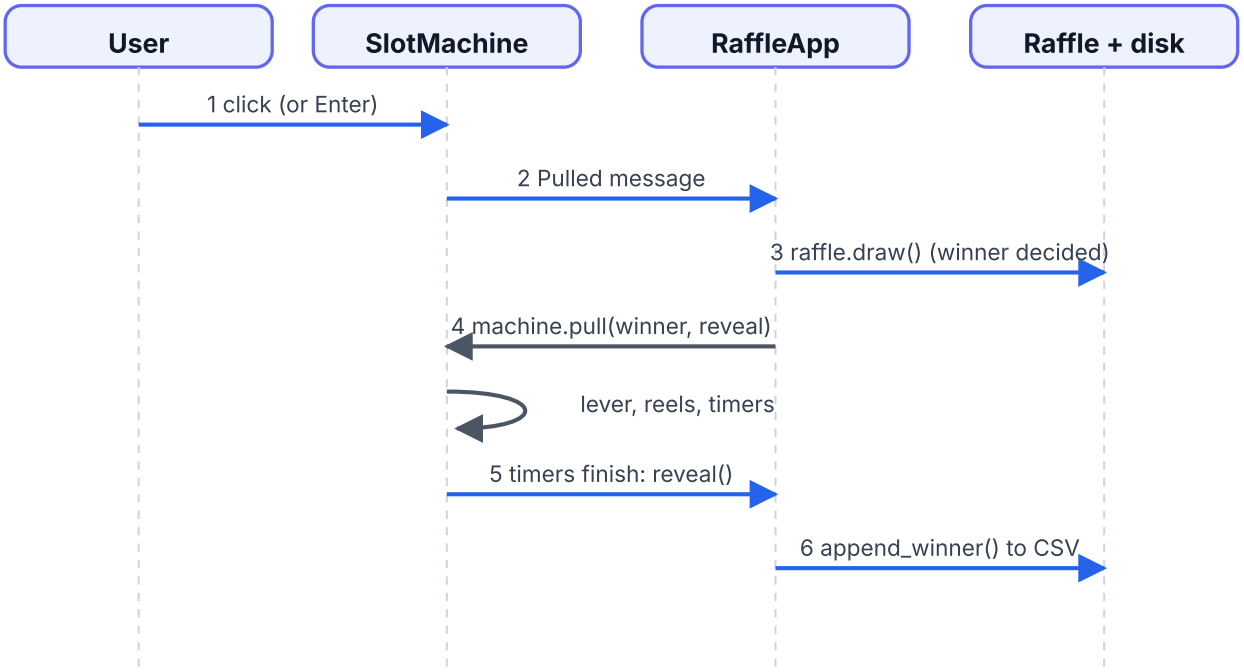
Total time from pull to reveal is about **3.8 seconds** (computed from the same formula the code uses). With `--fast` the delays shrink to 20 ms and the reels have 3 and 5 rows, which makes automated tests quick.

11.2 The lever and the mouse

src/raffle_cli/slot.py · SlotMachine.pull

```python
def pull(self, winner: Winner, on_done: Callable[[], None]) -> None:
    """Pull the lever; `on_done` runs once both reels have landed on `winner`."""
    self._on_done = on_done
    step = 0.02 if self.fast else 0.09
    self._set_lever(1)
    self.set_timer(step, lambda: self._set_lever(2))
    self.set_timer(2 * step, lambda: self._start_reels(winner))
    self.set_timer(3 * step, lambda: self._set_lever(1))
    self.set_timer(4 * step, lambda: self._set_lever(0))
```

The lever has three states (up, halfway, pulled), each just a different row for the red ball. `pull()` schedules them one after another, then starts the reels at the bottom of the pull. Clicking works because Textual calls a widget's `on_click` method for mouse clicks; we post a `Pulled` message, and the app (`on_slot_machine_pulled`) draws. **Keyboard and mouse run exactly the same code path**, so there is nothing extra to test or maintain.

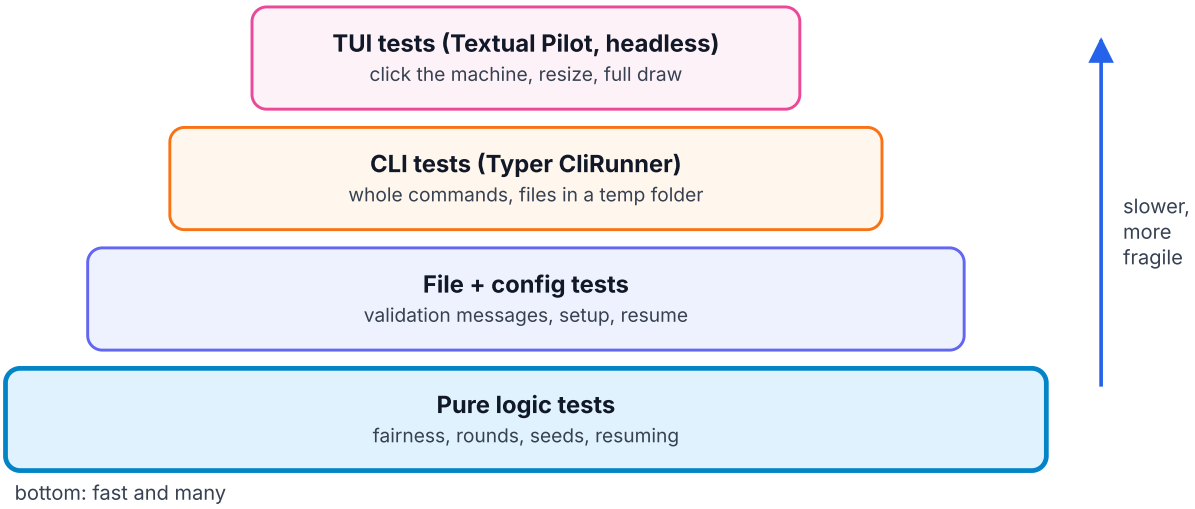


Technical decision: draw the machine as one text widget instead of many small widgets

Alternative: build the machine from Textual containers and labels (one label per row, one container per reel). It would be easier to style individually, but harder to animate in sync and to keep the lever aligned. Returning one `Text` from `render()` keeps every frame consistent and the whole machine is ~150 lines. **Trade-off:** character-art needs care with widths: that is why every line is padded to the same length and the cell width is clamped to 8–12 characters (long names are cut with "…").

12 Testing

There are **43 automated tests**: 13 for the engine, 16 for files and config, 14 for commands, fireworks and the Textual app. Run them with `uv run pytest`. A test is a function that sets up a situation, runs the code, and `assert`s what must be true. If any assertion is false, the test fails and tells you which.



The pyramid explains the structure: lots of fast tests at the bottom for pure logic (milliseconds, no files, no screen), fewer higher-level tests on top that exercise everything together. Because we split the engine from the interfaces in chapter 2, the most important rules can be tested with no screen at all.

tests/test_raffle.py · your rule “more prizes than people”

```python
def test_more_prizes_than_participants_are_shared_out_in_rounds():
    prizes = [Prize("a", 6), Prize("b", 4)]  # 10 prizes, 5 people
    winners = run_all(Raffle(PEOPLE, prizes, rng=random.Random(2)))
    assert len(winners) == 10
    assert set(Counter(w.full_name for w in winners).values()) == {2}  # fair: exactly two each
```

tests/test_raffle.py · reproducibility

```python
def test_same_seed_same_result():
    a = run_all(Raffle(PEOPLE, [Prize("a", 3)], rng=random.Random(42)))
    b = run_all(Raffle(PEOPLE, [Prize("a", 3)], rng=random.Random(42)))
    assert a == b
```

tests/test_cli.py · driving the real Textual app headlessly

```python
def test_tui_lever_click_draws_and_layout_is_responsive():
    from raffle_cli.slot import SlotMachine
    from raffle_cli.tui import RaffleApp

    people = [Participant("Ada", "Lovelace"), Participant("Guido", "van Rossum")]
    raffle = Raffle(people, [Prize("Mug", 2)], rng=random.Random(0))
    saved = []

    async def go():
        app_ = RaffleApp(raffle, saved.append, seed=0, fast=True, show_fireworks=False)
        async with app_.run_test(size=(140, 40)) as pilot:
            main = app_.query_one("#main")
            assert main.has_class("wide")
            await pilot.click(SlotMachine)  # mouse pulls the lever
            await pilot.pause(1.0)
```

```
            assert len(saved) == 1
            machine = app_.query_one(SlotMachine)
            assert [r.landed for r in machine.reels] == [True, True]
            assert [r.strip[r.pos] for r in machine.reels] == [saved[0].name, saved[0].lastname]
            assert machine.lever == 0
            await pilot.resize_terminal(90, 30)
            await pilot.pause()
            assert not main.has_class("wide")

    asyncio.run(go())
```

Technical decision: pytest + Textual's Pilot, without pytest-asyncio

Textual provides `app.run_test()`, which runs the app without a real terminal and gives you a **Pilot** to press keys, click widgets and resize the "terminal". The test body is `async`, so we wrap it with `asyncio.run(go())` inside a normal test function. **Alternative:** the `pytest-asyncio` plugin, which is one more dependency to explain. **Trade-off:** a few `pause()` calls to wait for timers; `--fast` keeps them short.

What tests cannot tell you

All of these tests pass without anybody looking at the screen. The fonts, how the lever looks in your terminal, and whether the fireworks *feel* good still need your own eyes. Run the program for real before an event.

13 Technical decisions, in one table

The same decisions explained in the chapters, as a quick reference. *Trade-off* is what you give up.

Decision	Alternative	Why	Trade-off
Engine in a pure module (`raffle.py`)	logic inside the UI code	testable, shared by both interfaces	one more file to learn
Frozen dataclasses	dicts / Pydantic	typo-proof, comparable, immutable	no automatic parsing
Prizes as a list of units	counters per prize	fewer special cases	list grows with total prizes (tiny)
Draw by minimal `(wins, holds_item)`	random with swap prompt	fair, explainable, never loops	swap feature dropped
People identified by index	by name	duplicate names allowed	needs a duplicate warning
Seed shown, `Random(seed)`	`SystemRandom`	auditable, reproducible	not secret (fine for swag)
Separate cosmetic RNG	reuse the raffle's RNG	animation can't change winners	two generators to remember
Decide first, save on reveal	save at draw time	file never spoils the reveal	quitting mid-spin redraws
Append winners one by one	write all at the end	crash-safe	many small writes (tiny)
Validate with all problems listed	fail on first error	fix everything in one go	slightly more code
Never auto-repair user files	offer defaults on error	cannot overwrite real data	user fixes by hand
Paths relative to config file	relative to the shell's folder	works from anywhere	slightly surprising for some
Pydantic only for config	manual dict checks	clear errors for little code	a dependency
Typer callback with `invoke_without_command`	explicit `run` subcommand	keeps plain `raffle-cli`	options precede subcommands
Textual as optional extra, lazy import	always installed	light default install	extra install step
Own fireworks simulation	curses / library	portable, tiny, educational	we maintain the physics
One simulation, two renderers	two implementations	no duplicate logic	renderer returns `Text` only
Rich `Live(screen=True)`	print frames and clear	flicker-free, terminal restored	needs a real terminal
Priority binding for Enter	make tables unfocusable	one line, key always works	shadows widget Enter
CSS class flips layout at 120 columns	two separate screens	one layout, tiny code	fixed breakpoint
Slot machine = one custom widget	many small widgets	frames stay consistent	care with text widths
Plain text names, no ASCII-art font	big figlet letters	long names and accents break art	smaller names
Timers instead of `sleep` loops	blocking animation	UI stays responsive	callbacks are harder to read
`asyncio.run` in tests	pytest-asyncio	no extra plugin	a little boilerplate
Python ≥ 3.10, uv + lockfile	newer only / pip freeze	wide support, reproducible installs	older syntax limits

14 Things that went wrong (and what they taught)

Real projects do not go in a straight line. These are the actual detours, because debugging stories teach more than polished code.

What happened	Cause	Lesson
Pressing Enter in the Textual app did nothing	The focused table handled Enter first	Events go to the focused widget first; use `priority=True`. Tests that simulate keys find this in seconds.
The “install the tui extra” message was cut to `pip install 'raffle-cli'`	Rich read `[tui]` as markup and swallowed it	Escape anything user-visible that might contain brackets (`rich.markup.escape`).
`uvx ... --tui` failed on a fresh machine	`uvx` does not install optional extras unless asked	Document the exact command (`--with textual`) and make the error say it.
Same seed, different winners in the two interfaces	Animation consumed numbers from the raffle's generator	Separate randomness that must be reproducible from randomness that is decoration.
Big ASCII-art names looked impressive then were dropped	Long names, accents and non-Latin letters either overflow or render blank	Features that need width handling for every possible input are expensive. Simpler plain text wins in a terminal.
The first idea for the spin showed one name at a time	We wanted “a literal spin”	Moving to reels (a list and a pointer) made the effect more convincing *and* simpler than the old code.

15 Exercises

This is where you take the code back. They are ordered from easiest to hardest. For each, write a test first (or at least run the program and look).

#	Exercise	Where to look	Hint
1	Change the fireworks colours and make them last longer	`fireworks.py` `PALETTE`, `ui.py` seconds	Run `raffle-cli fireworks --seconds 20` as you tweak.
2	Add a new `--order` value, `alpha` (prizes A to Z)	`raffle.py` `ORDERS`, `order_prizes`	One `elif` and a test. `sort(key=lambda p: p.item.lower())`.
3	Print a sentence at the end: "N winners, M prizes"	`ui.show_results`	You have `raffle.winners` and `raffle.remaining`.
4	Add an optional `email` column to participants that is ignored by the draw	`files.py`, `raffle.py`	The header check is strict: decide how to allow extra columns and test both cases.
5	Add a `--exclude "Name Lastname"` option for people who are present but not eligible (organisers)	`cli.py` `build_raffle`	Filter the participant list before creating the `Raffle`.
6	Export a Markdown results file next to `winners.csv`	new function in `files.py`	Call it after the last prize in `main`.
7	Add a third reel that shows the prize in the slot machine	`slot.py`	The reel list is built for 2; generalise it. Careful with total width and the CSS dock width (52).
8	Add a `weight` column so some people get more chances	`raffle.py` `draw`	Think about fairness first: how does it interact with "everyone wins once before anyone wins twice"? Write that rule down before coding.

16 Glossary and how to run everything

Run it

Without installing (macOS / Linux):

```bash / zsh
R="git+https://github.com/andoresuperesu/raffle-cli"
uvx --from $R raffle-cli setup
uvx --from $R --with textual raffle-cli --tui
```

PowerShell:

```PowerShell
$R = "git+https://github.com/andoresuperesu/raffle-cli"
uvx --from $R raffle-cli setup
uvx --from $R --with textual raffle-cli --tui
```

From a clone, for development:

```any shell
git clone https://github.com/andoresuperesu/raffle-cli
cd raffle-cli
uv sync
uv run pytest
uv run raffle-cli --config examples/config.yaml --tui
```

Known limits

- Developed and tested on Linux. On Windows use Windows Terminal; the box-drawing characters and mouse input are unverified there.
- The tests prove behaviour, not looks: check the slot machine and fireworks visually before an event.
- The 32-bit seed makes draws reproducible, not secret (see chapter 7).

Glossary

Callback	A function you hand to other code so it can call you back later (`on_winner` , `reveal`).
CLI	Command-line interface: a program you control by typing a command.
Dataclass	A class meant to hold data; Python generates the boring methods.
Entry point	The function a command like `raffle-cli` calls; declared in `pyproject.toml` .
Extra	Optional dependencies installed with `pkg[name]` .
Frozen	An object that cannot be modified after creation.
Pilot	Textual's test driver that presses keys and clicks in a headless app.
Pseudo-random	Looks random but is computed from a seed, so it can be repeated.

Pure function / module	Only computes from its inputs: no printing, no files, no clock.
Seed	The starting number of a pseudo-random generator.
Separation of concerns	Splitting a program so each part has a single job.
Tick	One step of a simulation or animation.
TUI	Text user interface: a full-screen app drawn with characters.
Virtual environment	A private folder of installed packages for one project (`.venv`).
Widget	A reusable piece of a user interface (a table, a label, the slot machine).

Where to read more

- Typer: typer.tiangolo.com · Rich: rich.readthedocs.io · Textual: textual.textualize.io
- Pydantic: docs.pydantic.dev · pytest: docs.pytest.org · uv: docs.astral.sh/uv
- The Python tutorial on classes, exceptions and modules: docs.python.org/3/tutorial